

Project: Jamming e traffico veicolare

automi cellulari stocastici, transizioni di fase e onde di stop-and-go

1. Obiettivi della dispensa

Questa dispensa introduce il modello di Nagel-Schreckenberg come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. formalizzare il traffico veicolare come automa cellulare stocastico a tempo discreto;
2. derivare il diagramma fondamentale del traffico, cioè la relazione tra flusso e densità;
3. identificare la transizione di fase tra regime fluido e regime congestionato;
4. mostrare come le onde di stop-and-go emergano spontaneamente da regole microscopiche locali in assenza di incidenti o ostacoli;
5. introdurre osservabili quantitative per misurare l'ordine, la congestione e la struttura spaziale del traffico;
6. confrontare il modello stocastico con la versione deterministica e discutere il ruolo del rumore.

Dal punto di vista del corso, questo modello è particolarmente interessante perché mostra come una transizione di fase collettiva possa emergere da regole di aggiornamento individuali molto semplici, in un contesto applicativo immediato e visivamente intuitivo.

2. Motivazione: cosa è il jamming e dove lo incontriamo

Il jamming è il fenomeno per cui un sistema di agenti in moto si blocca collettivamente, anche in assenza di ostacoli fisici o di cause esterne evidenti.

Esempi concreti e quotidiani:

- **Autostrada.** Si è in coda da venti minuti. Quando finalmente si esce dalla congestione, non c'è nessun incidente, nessun cantiere, nessuna rampa. La coda è nata spontaneamente da una piccola fluttuazione della densità

del traffico, amplificata dalle frenate a catena. Questo e' esattamente il fenomeno che il modello descrive.

- **Coda alla cassa del supermercato.** Quando ci sono tanti clienti, anche rallentamenti minimi del cassiere producono code lunghissime. E' il limite di saturazione che abbiamo visto nella teoria delle code, ma qui la struttura spaziale conta.
- **Pedoni in un corridoio stretto.** Se la densita' di persone supera una soglia critica, il flusso si interrompe anche senza ostacoli: le persone si bloccano a vicenda. Lo stesso fenomeno avviene durante l'evacuazione di edifici.
- **Rete internet.** I pacchetti di dati si accodano nei router quando il traffico e' elevato. Le code si formano e si propagano verso monte esattamente come nel traffico veicolare.
- **Formiche lungo un sentiero.** Colonie di formiche mostrano transizioni tra flusso libero e congestione al variare della densita', con struttura spaziale analoga a quella del traffico.

In tutti questi casi la struttura e' la stessa: agenti in moto su uno spazio discreto o continuo, regole di interazione locale, transizione tra fase fluida e fase congestionata al variare della densita'.

3. Il modello di Nagel-Schreckenberg

3.1 Struttura della strada

Si considera una strada a una corsia rappresentata come un reticolo periodico di L celle. Ogni cella puo' contenere al piu' un veicolo.

Con condizioni periodiche al contorno, la strada e' topologicamente un anello: il veicolo che esce dall'ultima cella rientra dalla prima. Questo evita effetti di bordo e permette di studiare il comportamento stazionario.

Il numero di veicoli sulla strada e' N , fisso per tutta la simulazione. La densita' di traffico e':

$$\rho = \frac{N}{L} \in (0, 1].$$

Per $\rho \rightarrow 0$ la strada e' quasi vuota; per $\rho \rightarrow 1$ ogni cella e' occupata e il traffico e' completamente bloccato.

3.2 Variabili di stato

Ogni veicolo i e' caratterizzato da:

- una posizione $x_i \in \{0, 1, \dots, L-1\}$;
- una velocita' $v_i \in \{0, 1, \dots, v_{\max}\}$.

La velocità è un intero. Il parametro $v_{\max}$ è la velocità massima consentita. Nella versione originale del modello si usa spesso $v_{\max} = 5$, che in unità fisiche corrisponde a circa 135 km/h su autostrada.

Lo spazio libero davanti al veicolo i è:

$$d_i = x_{i+1} - x_i - 1,$$

dove x_{i+1} è la posizione del veicolo immediatamente avanti, e la sottrazione di 1 tiene conto che la cella occupata dal veicolo precedente non è disponibile. Le distanze si calcolano modulo L per le condizioni periodiche.

3.3 Le quattro regole di aggiornamento

Il cuore del modello sono quattro regole applicate in sequenza a tutti i veicoli in parallelo (aggiornamento sincrono) a ogni passo temporale:

Regola 1 – Accelerazione:

$$v_i \leftarrow \min(v_i + 1, v_{\max}).$$

Ogni veicolo tende ad aumentare la propria velocità di un'unità, fino al massimo consentito. Modella la tendenza naturale degli automobilisti ad accelerare quando hanno spazio.

Regola 2 – Frenata per sicurezza:

$$v_i \leftarrow \min(v_i, d_i).$$

Se il veicolo si troverebbe a collidere con quello davanti, frena il necessario per mantenere la distanza di sicurezza. Questa regola garantisce che non ci siano collisioni.

Regola 3 – Rumore (frenata casuale):

$$v_i \leftarrow \max(v_i - 1, 0) \quad \text{con probabilità } p.$$

Con probabilità p il veicolo rallenta di un'unità, anche se non c'è nessun ostacolo davanti. Questa regola cattura l'imperfezione del comportamento umano: distrazione, reazione ritardata, cambio di marcia, incertezza.

Regola 4 – Movimento:

$$x_i \leftarrow x_i + v_i.$$

Ogni veicolo avanza della propria velocità corrente.

Interpretazione fisica. Le prime due regole sono deterministiche e riflettono il comportamento razionale di un automobilista ideale: accelera se può, frena se deve. La terza regola introduce il rumore: anche un automobilista attento rallenta a volte senza ragione apparente. È proprio questo rumore che, come vedremo, è responsabile delle onde di stop-and-go.

3.4 Il caso deterministico $p = 0$

Senza rumore, il modello e' completamente deterministico. A bassa densita', ogni veicolo raggiunge rapidamente $v_{\max}$ e scorre liberamente. A densita' piu' alta, i veicoli si avvicinano e la velocita' media decresce. Il sistema raggiunge sempre uno stato stazionario ordinato, senza fluttuazioni.

3.5 Il caso stocastico $p > 0$

Con rumore, il comportamento e' qualitativamente diverso. Anche a densita' moderate, una piccola frenata casuale di un veicolo costringe il veicolo dietro a frenare a sua volta, il quale costringe il successivo, e cosi' via. Si forma un'onda di frenata che si propaga all'indietro (in direzione opposta al moto dei veicoli) anche dopo che il veicolo che ha iniziato la perturbazione ha ripreso velocita'.

Questo e' il meccanismo delle onde fantasma (phantom traffic jams): onde senza causa apparente, che si spostano controcorrente rispetto al flusso.

4. Osservabili macroscopiche

Per analizzare la dinamica servono osservabili quantitative.

4.1 Velocita' media

$$\bar{v}(t) = \frac{1}{N} \sum_{i=1}^N v_i(t).$$

In regime fluido $\bar{v}$ e' vicina a $v_{\max}$; in regime congestionato scende verso zero.

4.2 Flusso

Il flusso misura quanti veicoli attraversano una sezione della strada per unita' di tempo:

$$q = \rho \bar{v}.$$

Questa relazione e' l'analogo della legge di continuita': il flusso e' il prodotto di densita' e velocita' media.

4.3 Diagramma fondamentale

Il diagramma fondamentale del traffico e' il grafico di q in funzione di ρ .

Regime fluido (bassa densita'). I veicoli scorrono a velocita' vicina a $v_{\max}$, quindi $q \approx \rho v_{\max}$: il flusso cresce linearmente con la densita'.

Regime congestionato (alta densita'). I veicoli si bloccano a vicenda. Quando la densita' sale, la velocita' media crolla e il flusso diminuisce.

Flusso massimo. Esiste una densita' critica ρ_c in cui il flusso e' massimo. Questa densita' separa il regime fluido dal regime congestionato ed e' uno dei parametri fondamentali del modello.

Il diagramma fondamentale ha quindi una forma a campana o a triangolo, con un massimo in ρ_c . La forma precisa dipende da $v_{\max}$ e p .

Esempio concreto. Su un'autostrada reale, il flusso massimo e' di circa 2000 veicoli per corsia per ora, raggiunto a una densita' di circa 25-30 veicoli/km. Oltre questa soglia, aggiungere veicoli riduce il flusso invece di aumentarlo: la strada "satura". Il modello di Nagel-Schreckenberg riproduce questa forma qualitativa in modo molto accurato.

4.4 Diagramma spazio-tempo

Il diagramma spazio-tempo e' la rappresentazione grafica delle traiettorie dei veicoli: sull'asse orizzontale il tempo, sull'asse verticale lo spazio (posizione sulla strada). Ogni veicolo traccia una linea.

In regime fluido le linee sono quasi parallele e inclinate (tutti scorrono alla stessa velocita'). In regime congestionato si vedono chiaramente le onde di stop-and-go: regioni di alta densita' che si spostano all'indietro (inclinate in senso opposto al moto).

Questo grafico e' uno degli strumenti visivi piu' potenti del progetto.

4.5 Parametro d'ordine

Un parametro d'ordine semplice per distinguere i due regimi e' la velocita' media normalizzata:

$$\Phi = \frac{\bar{v}}{v_{\max}} \in [0, 1].$$

Per $\Phi \approx 1$ il sistema e' in regime fluido; per $\Phi \approx 0$ il sistema e' completamente congestionato.

4.6 Fluttuazioni della velocita'

La deviazione standard della velocita':

$$\sigma_v = \sqrt{\frac{1}{N} \sum_{i=1}^N (v_i - \bar{v})^2}$$

e' massima vicino alla transizione, dove coesistono regioni fluide e regioni congestionate. Questo e' analogo alla suscettivita' nei sistemi fisici vicino a una transizione di fase.

5. La transizione di fase

5.1 Regime fluido e regime congestionato

Il modello mostra una transizione tra due regimi qualitativamente diversi:

- **Regime fluido** (bassa ρ): i veicoli si muovono quasi liberamente, le interazioni sono rare, la velocita' media e' alta.
- **Regime congestionato** (alta ρ): i veicoli si bloccano a vicenda, si formano code, la velocita' media e' bassa.

5.2 Il ruolo del rumore

Il rumore p abbassa la densita' critica ρ_c e riduce il flusso massimo. Con $p = 0$ la transizione avviene a densita' piu' alta e il flusso massimo e' maggiore. Con p vicino a 1 la transizione avviene gia' a bassa densita' e il traffico e' sempre molto congestionato.

Questo e' il risultato piu' sorprendente del modello: **le onde di stop-and-go non richiedono una causa esterna**. Bastano la densita' sufficientemente alta e una piccola probabilita' di frenata casuale per far emergere spontaneamente la congestione.

Esempio concreto. Su un'autostrada reale si osservano code che si formano e si dissolvono senza che ci sia nessun incidente o rallentamento iniziale. Il modello spiega esattamente questo: la congestione e' un fenomeno collettivo che emerge dalla combinazione di alta densita' e imperfezione del comportamento individuale.

5.3 Isteresi

In molti sistemi fisici reali, la transizione tra fase fluida e fase congestionata mostra isteresi: il traffico che entra in congestione da uno stato fluido si comporta diversamente da quello che esce dalla congestione. Questo fenomeno non e' presente nel modello base di Nagel-Schreckenberg ma puo' emergere in varianti piu' ricche.

6. Varianti del modello base

6.1 Limite di velocita' variabile

Si puo' assegnare a ciascun veicolo un $v_{\max}$ diverso, per modellare l'eterogeneita' degli automobilisti (auto veloci, camion, veicoli lenti). L'eterogeneita' tende ad

aumentare la congestione perche' i veicoli veloci sono costretti a seguire quelli lenti.

6.2 Inizio lento (slow-to-start)

Una variante molto studiata e' il modello "slow-to-start": un veicolo che era fermo ($v = 0$) non accelera subito, ma rimane fermo per un altro passo con probabilita' q . Questa modifica produce una transizione di fase piu' netta e maggiore isteresi, piu' vicina ai dati empirici.

6.3 Strade a due corsie

Si puo' aggiungere una seconda corsia con sorpassi. Ogni veicolo puo' cambiare corsia se il veicolo davanti e' troppo lento e la corsia adiacente e' libera. Questa estensione introduce interazioni laterali e produce fenomeni piu' ricchi, come la formazione di cluster organizzati.

6.4 Incroci e semafori

Si possono introdurre incroci regolati da semafori, modellando il traffico urbano. Il semaforo e' una perturbazione periodica che interagisce con le onde di stop-and-go in modo non banale.

7. Connessione con altri modelli del corso

Il modello di Nagel-Schreckenberg si colloca in modo molto naturale nel contesto di altri modelli del corso.

Come il modello di Vicsek, descrive una transizione collettiva da disordine a ordine (o viceversa) in un sistema di agenti in moto. La differenza e' che qui l'ordine e' un regime fluido (tutti si muovono nella stessa direzione alla stessa velocita') mentre il disordine e' la congestione.

Come le dinamiche replicative, ha una transizione di fase al variare di un parametro di controllo (la densita') e mostra come piccole perturbazioni microscopiche si amplifichino a scala macroscopica.

Come il modello di Vicsek, l'aggiornamento e' sincrono, le interazioni sono locali, e le osservabili macroscopiche emergono da regole microscopiche semplici.

8. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande precise.

1. Come dipende il flusso massimo da $v_{\max}$ e da p ?

2. A quale densita' si trova il flusso massimo, e come cambia con p ?
3. Come si propagano le onde di stop-and-go? Qual e' la loro velocita' e direzione?
4. Il rumore e' necessario per produrre congestione, o anche il caso deterministico mostra jamming?
5. Come cambia il diagramma fondamentale al variare della taglia del sistema L ?
6. In che modo l'eterogeneita' dei veicoli modifica la transizione di fase?

9. Pseudocodice del modello

9.1 Input

- lunghezza della strada L
- numero di veicoli N (o densita' $\rho = N/L$)
- velocita' massima $v_{\max}$
- probabilita' di frenata casuale p
- numero di passi temporali T
- numero di passi di transiente da scartare T_{burn}

9.2 Pseudocodice

1. inizializza le posizioni x_i distribuendo i veicoli uniformemente sulla strada;
2. inizializza le velocita' $v_i = v_{\max}$ per tutti i veicoli;
3. per $t = 1, \dots, T$:
 - per ogni veicolo i (calcola in parallelo):
 - calcola lo spazio libero $d_i = x_{i+1} - x_i - 1$ (modulo L)
 - applica la regola 1: $v_i \leftarrow \min(v_i + 1, v_{\max})$
 - applica la regola 2: $v_i \leftarrow \min(v_i, d_i)$
 - applica la regola 3: con probabilita' p , $v_i \leftarrow \max(v_i - 1, 0)$
 - per ogni veicolo i :
 - applica la regola 4: $x_i \leftarrow (x_i + v_i) \bmod L$
 - se $t > T_{\text{burn}}$:
 - calcola e salva $\bar{v}(t)$, $q(t)$, $\sigma_v(t)$
 - salva le posizioni per il diagramma spazio-tempo

9.3 Nota sull'aggiornamento sincrono

E' fondamentale che le regole 1, 2 e 3 siano applicate a tutti i veicoli prima di eseguire i movimenti (regola 4). Se si aggiornasse un veicolo alla volta, le regole di sicurezza verrebbero violate perche' un veicolo userebbe la posizione gia' aggiornata del veicolo davanti invece di quella al tempo t .

10. Schema del laboratorio

10.1 Laboratorio 1 - Implementazione e visualizzazione

Obiettivo

Implementare il modello e osservare qualitativamente la transizione tra regime fluido e congestionato.

Attività

1. fissare $L = 100$, $v_{\max} = 5$, $p = 0.3$;
2. simulare per $\rho = 0.1$ (bassa densità) e $\rho = 0.5$ (alta densità);
3. visualizzare il diagramma spazio-tempo per i due regimi;
4. calcolare $\bar{v}$ e σ_v per i due casi.

Domande guida

- nel diagramma spazio-tempo a bassa densità, le traiettorie sono quasi parallele o mostrano interruzioni?
- ad alta densità, si vedono chiaramente le onde che si propagano all'indietro?
- la velocità media è stabile o fluttua nel tempo?

Output richiesto

- codice sorgente;
- diagrammi spazio-tempo per i due regimi;
- traiettorie temporali di $\bar{v}(t)$;
- commento qualitativo sulla differenza tra i due regimi.

10.2 Laboratorio 2 - Il diagramma fondamentale

Obiettivo

Costruire il diagramma fondamentale $q(\rho)$ e identificare la densità critica.

Attività

1. fissare $L = 200$, $v_{\max} = 5$, $p = 0.3$;
2. variare ρ su una griglia da 0.05 a 1.0;
3. per ogni ρ , simulare per $T = 2000$ passi con $T_{\text{burn}} = 500$;
4. stimare $q = \rho \bar{v}$ e costruire il grafico $q(\rho)$.

Domande guida

- il flusso massimo è a quale valore di ρ ?
- il diagramma è simmetrico o asimmetrico attorno al massimo?
- come cambia la forma al variare di p ?

Output richiesto

- grafico del diagramma fondamentale per $p = 0$, $p = 0.1$, $p = 0.3$;
- tabella con densita' critica e flusso massimo per i tre valori di p ;
- commento sul ruolo del rumore.

10.3 Laboratorio 3 - Onde di stop-and-go e ruolo del rumore

Obiettivo

Studiare la formazione e la propagazione delle onde di stop-and-go al variare di p .

Attivita'

1. fissare $L = 200$, $v_{\max} = 5$, $\rho = 0.3$ (densita' moderata);
2. simulare per $p = 0$ (deterministico) e $p = 0.3$;
3. confrontare i diagrammi spazio-tempo;
4. stimare la velocita' di propagazione delle onde nel caso stocastico.

Domande guida

- nel caso deterministico si formano onde?
- nel caso stocastico, in quale direzione si propagano le onde rispetto al moto dei veicoli?
- la velocita' delle onde dipende dalla densita'?

Output richiesto

- diagrammi spazio-tempo a confronto;
- stima della velocita' delle onde;
- discussione del meccanismo di amplificazione delle fluttuazioni.

10.4 Laboratorio 4 - Analisi parametrica e transizione di fase

Obiettivo

Studiare come il parametro d'ordine $\Phi = \bar{v}/v_{\max}$ varia con ρ e p .

Attivita'

1. costruire la heatmap di $\Phi(\rho, p)$ su una griglia di valori;
2. identificare la regione di transizione nel piano (ρ, p) ;
3. misurare le fluttuazioni σ_v e verificare se sono massime vicino alla transizione;
4. confrontare con il caso $v_{\max} = 1$ (modello elementare).

Domande guida

- esiste una curva nel piano (ρ, p) che separa il regime fluido da quello congestionato?
- le fluttuazioni sono effettivamente massime vicino alla transizione?
- come cambia la transizione al variare di $v_{\max}$?

Output richiesto

- heatmap di $\Phi(\rho, p)$;
- grafico di $\sigma_v(\rho)$ per un valore fisso di p ;
- commento sulla struttura della transizione.

11. Il caso $v_{\max} = 1$: il modello elementare

Il caso $v_{\max} = 1$ e' particolarmente semplice e istruttivo. Ogni cella puo' essere vuota o occupata, e ogni veicolo si muove di una cella o rimane fermo.

Le quattro regole si semplificano enormemente:

- se la cella davanti e' libera: il veicolo si muove con probabilita' $1 - p$, rimane fermo con probabilita' p ;
- se la cella davanti e' occupata: il veicolo rimane fermo.

Questo e' formalmente equivalente al modello ASEP (Asymmetric Simple Exclusion Process), uno dei modelli piu' studiati della fisica statistica. L'ASEP ha una soluzione esatta e il suo diagramma fondamentale e' una parabola:

$$q(\rho) = \rho(1 - \rho)(1 - p).$$

Il flusso massimo e' raggiunto a $\rho = 1/2$ e vale $(1 - p)/4$.

Questo risultato e' molto utile didatticamente: fornisce un confronto analitico preciso con la simulazione.

12. Perche' questo e' un buon case study per il corso

Questo progetto e' particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, e' visivamente molto immediato: il diagramma spazio-tempo mostra in modo inequivocabile la differenza tra regime fluido e congestionato, e le onde di stop-and-go sono immediatamente riconoscibili.

Secondo, la struttura di aggiornamento sincrono e' semplice ma diversa dai modelli a tempo continuo: e' utile che gli studenti abbiano esperienza con entrambi i tipi di dinamica.

Terzo, il diagramma fondamentale e' una osservabile macroscopica che emerge in modo molto pulito dalla simulazione, permettendo un confronto diretto con il caso analitico $v_{\max} = 1$.

Quarto, il ruolo del rumore e' molto esplicito: il confronto tra $p = 0$ e $p > 0$ mostra in modo netto che la congestione spontanea e' un fenomeno stocastico.

Quinto, il modello e' computazionalmente leggerissimo: anche con $L = 1000$ e $N = 500$, una simulazione di $T = 10000$ passi e' istantanea, permettendo di fare analisi parametriche molto complete.

13. Conclusione

Il modello di Nagel-Schreckenberg mostra come un fenomeno quotidiano e familiare come la coda in autostrada possa essere spiegato con regole microscopiche semplicissime. Non servono incidenti, non servono ostacoli: basta una densita' sufficientemente alta e una piccola probabilita' di frenata casuale per generare onde di stop-and-go che si propagano all'indietro attraverso l'intero sistema.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- automi cellulari stocastici;
- aggiornamento sincrono su reticolo;
- diagramma fondamentale come osservabile macroscopica;
- transizione di fase collettiva;
- visualizzazione spazio-temporale;
- confronto tra caso deterministico e stocastico.

Il messaggio concettuale piu' importante e' che la congestione non e' un fallimento individuale ma un fenomeno collettivo emergente: nessun singolo automobilista "sbaglia", eppure il sistema produce code. E' esattamente il tipo di fenomeno che i metodi computazionali per modelli stocastici sono progettati per studiare.

14. Bibliografia minima

1. Nagel, K., and Schreckenberg, M. (1992). A Cellular Automaton Model for Freeway Traffic. *Journal de Physique I*, 2(12), 2221-2229.
2. Chowdhury, D., Santen, L., and Schadschneider, A. (2000). Statistical Physics of Vehicular Traffic and Some Related Systems. *Physics Reports*, 329(4-6), 199-329.
3. Helbing, D. (2001). Traffic and Related Self-Driven Many-Particle Systems. *Reviews of Modern Physics*, 73(4), 1067-1141.
4. Schadschneider, A., Chowdhury, D., and Nishinari, K. (2010). *Stochastic Transport in Complex Systems*. Elsevier.
5. Derrida, B. (1998). An Exactly Soluble Non-Equilibrium System: The Asymmetric Simple Exclusion Process. *Physics Reports*, 301(1-3), 65-83.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python il modello di Nagel-Schreckenberg e le analisi associate.

L'obiettivo non e' costruire un simulatore ottimizzato, ma fornire una guida leggibile che possa essere letta:

- come pseudocodice da chi usa altri linguaggi;
- come base quasi immediatamente eseguibile da chi conosce Python.

Il codice e' volutamente elementare:

- poche librerie;
- liste e cicli espliciti;
- funzioni corte;
- nomi leggibili.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

Non e' necessario usare numpy in una prima implementazione.

A.2 Inizializzazione dello stato

Lo stato del sistema e' rappresentato da due liste: le posizioni e le velocita' dei veicoli.

```
def initialize_traffic(L, N, v_max):
    if N > L:
        raise ValueError("Non possono esserci piu' veicoli che celle.")

    # distribuisce i veicoli uniformemente sulla strada
    positions = []
    step = L // N

    for i in range(N):
        positions.append((i * step) % L)

    # velocita' iniziali al massimo
    velocities = [v_max] * N
```

```
    return positions, velocities
```

Per una distribuzione casuale delle posizioni iniziali:

```
def initialize_traffic_random(L, N, v_max):
    all_cells = list(range(L))
    random.shuffle(all_cells)
    positions = sorted(all_cells[:N])
    velocities = [random.randint(0, v_max) for _ in range(N)]
    return positions, velocities
```

A.3 Calcolo degli spazi liberi

```
def compute_gaps(positions, L):
    N = len(positions)
    gaps = []

    for i in range(N):
        next_i = (i + 1) % N
        gap = (positions[next_i] - positions[i] - 1) % L
        gaps.append(gap)

    return gaps
```

Nota: il calcolo modulo L gestisce automaticamente le condizioni periodiche, incluso il caso in cui l'ultimo veicolo deve “guardare” il primo.

A.4 Un passo di aggiornamento

```
def nasch_step(positions, velocities, L, v_max, p):
    N = len(positions)
    gaps = compute_gaps(positions, L)

    new_velocities = velocities[:]

    # regole 1, 2, 3 applicate a tutti i veicoli (aggiornamento parallelo)
    for i in range(N):
        v = velocities[i]

        # regola 1: accelerazione
        v = min(v + 1, v_max)

        # regola 2: frenata per sicurezza
        v = min(v, gaps[i])

        # regola 3: frenata casuale
```

```

        if random.random() < p:
            v = max(v - 1, 0)

        new_velocities[i] = v

# regola 4: movimento (dopo aver aggiornato tutte le velocita')
new_positions = positions[:]

for i in range(N):
    new_positions[i] = (positions[i] + new_velocities[i]) % L

# riordina i veicoli per posizione crescente
pairs = sorted(zip(new_positions, new_velocities))
new_positions = [p for p, v in pairs]
new_velocities = [v for p, v in pairs]

return new_positions, new_velocities

```

Nota importante. Il riordinamento alla fine serve a mantenere la lista dei veicoli ordinata per posizione, il che semplifica il calcolo degli spazi liberi al passo successivo. Non cambia la fisica del modello.

A.5 Simulazione completa

```

def simulate_nasch(L, N, v_max, p, T, T_burn=200):
    positions, velocities = initialize_traffic(L, N, v_max)

    history_positions = []
    history_velocities = []
    mean_velocities = []
    std_velocities = []

    for t in range(T):
        positions, velocities = nasch_step(positions, velocities, L, v_max, p)

        if t >= T_burn:
            history_positions.append(positions[:])
            history_velocities.append(velocities[:])

            v_mean = sum(velocities) / N
            v_std = math.sqrt(sum((v - v_mean) ** 2 for v in velocities) / N)

            mean_velocities.append(v_mean)
            std_velocities.append(v_std)

    rho = N / L

```

```

avg_v = statistics.mean(mean_velocities) if mean_velocities else 0.0
flow = rho * avg_v

results = {
    "rho": rho,
    "avg_velocity": avg_v,
    "flow": flow,
    "mean_velocities": mean_velocities,
    "std_velocities": std_velocities,
    "history_positions": history_positions,
    "history_velocities": history_velocities
}

return results

```

A.6 Diagramma spazio-tempo

Il diagramma spazio-tempo e' la visualizzazione piu' importante del modello.

```

def plot_spacetime(history_positions, L, title="Diagramma spazio-tempo"):
    T = len(history_positions)

    plt.figure(figsize=(10, 6))

    for t, positions in enumerate(history_positions):
        for x in positions:
            plt.plot(t, x, "k.", markersize=1)

    plt.xlabel("tempo")
    plt.ylabel("posizione")
    plt.title(title)
    plt.xlim(0, T)
    plt.ylim(0, L)
    plt.show()

```

In alternativa, per una visualizzazione piu' compatta come immagine:

```

def spacetime_matrix(history_positions, L):
    T = len(history_positions)
    matrix = [[0] * L for _ in range(T)]

    for t, positions in enumerate(history_positions):
        for x in positions:
            matrix[t][x] = 1

    return matrix

```



```
def plot_spacetime_image(history_positions, L, title="Diagramma spazio-tempo"):
    matrix = spacetime_matrix(history_positions, L)

    plt.imshow(matrix, cmap="binary", aspect="auto", origin="lower")
    plt.xlabel("posizione")
    plt.ylabel("tempo")
    plt.title(title)
    plt.colorbar(label="occupazione")
    plt.show()
```

Esempio:

```
res = simulate_nasch(L=150, N=45, v_max=5, p=0.3, T=300, T_burn=50)
plot_spacetime_image(res["history_positions"], L=150,
                      title="NaSch: regime congestionato")
```

A.7 Diagramma fondamentale

```
def compute_fundamental_diagram(L, v_max, p, T=2000, T_burn=500,
                                rho_values=None):
    if rho_values is None:
        rho_values = [0.05 * k for k in range(1, 21)]

    flows = []
    avg_velocities = []

    for rho in rho_values:
        N = max(1, int(round(rho * L)))
        res = simulate_nasch(L=L, N=N, v_max=v_max, p=p,
                              T=T, T_burn=T_burn)
        flows.append(res["flow"])
        avg_velocities.append(res["avg_velocity"])

    return rho_values, flows, avg_velocities

def plot_fundamental_diagram(rho_values, flows, label="", title="Diagramma fondamentale"):
    plt.plot(rho_values, flows, label=label, marker="o", markersize=4)
    plt.xlabel("densita' rho")
    plt.ylabel("flusso q")
    plt.title(title)
    if label:
        plt.legend()
    plt.show()
```

Confronto tra diversi valori di p :

```

L = 200
v_max = 5
rho_values = [0.05 * k for k in range(1, 21)]

for p_val in [0.0, 0.1, 0.3, 0.5]:
    rho_vals, flows, _ = compute_fundamental_diagram(
        L=L, v_max=v_max, p=p_val,
        T=2000, T_burn=500,
        rho_values=rho_values
    )
    plt.plot(rho_vals, flows, label=f"p = {p_val}", marker="o", markersize=3)

plt.xlabel("densita' rho")
plt.ylabel("flusso q")
plt.title("Diagramma fondamentale per diversi valori di p")
plt.legend()
plt.show()

```

A.8 Caso analitico $v_{\max} = 1$ (ASEP)

```

def asep_flow(rho, p):
    return rho * (1.0 - rho) * (1.0 - p)

def plot_asep_comparison(p, L=200, T=2000, T_burn=500):
    rho_values = [0.05 * k for k in range(1, 21)]

    # simulazione
    _, flows_sim, _ = compute_fundamental_diagram(
        L=L, v_max=1, p=p, T=T, T_burn=T_burn,
        rho_values=rho_values
    )

    # analitico
    flows_analytic = [asep_flow(rho, p) for rho in rho_values]

    plt.plot(rho_values, flows_sim, "o-", label="simulazione")
    plt.plot(rho_values, flows_analytic, "--", label="analitico")
    plt.xlabel("densita' rho")
    plt.ylabel("flusso q")
    plt.title(f"ASEP (v_max=1, p={p}): simulazione vs analitico")
    plt.legend()
    plt.show()

```

Questo confronto e' didatticamente molto utile: verifica che la simulazione riproduca la formula analitica esatta nel caso $v_{\max} = 1$.

A.9 Parametro d'ordine e fluttuazioni

```
def order_parameter(mean_velocities, v_max):
    avg = statistics.mean(mean_velocities) if mean_velocities else 0.0
    return avg / v_max

def velocity_fluctuations(std_velocities):
    return statistics.mean(std_velocities) if std_velocities else 0.0
```

A.10 Heatmap del parametro d'ordine nel piano (rho, p)

```
def compute_phase_diagram(L, v_max, T=1500, T_burn=300,
                           rho_values=None, p_values=None):
    if rho_values is None:
        rho_values = [0.1 * k for k in range(1, 11)]
    if p_values is None:
        p_values = [0.1 * k for k in range(1, 11)]

    phi_matrix = []

    for p in p_values:
        row = []
        for rho in rho_values:
            N = max(1, int(round(rho * L)))
            res = simulate_nasch(L=L, N=N, v_max=v_max, p=p,
                                T=T, T_burn=T_burn)
            phi = order_parameter(res["mean_velocities"], v_max)
            row.append(phi)
        phi_matrix.append(row)

    return rho_values, p_values, phi_matrix

def plot_phase_diagram(rho_values, p_values, phi_matrix):
    plt.imshow(
        phi_matrix,
        origin="lower",
        aspect="auto",
        extent=[rho_values[0], rho_values[-1],
                p_values[0], p_values[-1]],
        cmap="RdYlGn",
        vmin=0.0,
        vmax=1.0
    )
    plt.colorbar(label="parametro d'ordine Phi")
```

```

plt.xlabel("densita' rho")
plt.ylabel("probabilita' di frenata p")
plt.title("Diagramma di fase: fluido (verde) vs congestionato (rosso)")
plt.show()

```

A.11 Velocita' di propagazione delle onde

Per stimare la velocita' delle onde di stop-and-go, si puo' osservare come si spostano le regioni di alta densita' nel diagramma spazio-tempo.

Una stima semplice consiste nel calcolare lo spostamento medio delle celle vuote tra passi successivi:

```

def estimate_wave_speed(history_positions, L, sample_steps=50):
    N_steps = len(history_positions)
    if N_steps < sample_steps + 1:
        return None

    displacements = []

    for t in range(N_steps - sample_steps, N_steps - 1):
        occupied_t = set(history_positions[t])
        occupied_t1 = set(history_positions[t + 1])

        empty_t = set(range(L)) - occupied_t
        empty_t1 = set(range(L)) - occupied_t1

        if not empty_t or not empty_t1:
            continue

        avg_empty_t = sum(empty_t) / len(empty_t)
        avg_empty_t1 = sum(empty_t1) / len(empty_t1)

        # spostamento medio delle regioni vuote (cioe' delle onde)
        disp = (avg_empty_t1 - avg_empty_t) % L
        if disp > L / 2:
            disp -= L

        displacements.append(disp)

    if not displacements:
        return None

    return statistics.mean(displacements)

```

A.12 Variante slow-to-start

```
def nasch_slow_to_start_step(positions, velocities, was_stopped,
                             L, v_max, p, q_slow):
    N = len(positions)
    gaps = compute_gaps(positions, L)

    new_velocities = velocities[:]
    new_was_stopped = was_stopped[:]

    for i in range(N):
        v = velocities[i]

        # regola 1: accelerazione (ma se era fermo, rimane fermo con prob q_slow)
        if v == 0 and was_stopped[i]:
            if random.random() < q_slow:
                new_was_stopped[i] = True
            else:
                v = min(v + 1, v_max)
                new_was_stopped[i] = False
        else:
            v = min(v + 1, v_max)
            new_was_stopped[i] = False

        # regola 2: frenata per sicurezza
        v = min(v, gaps[i])

        # regola 3: frenata casuale
        if random.random() < p:
            v = max(v - 1, 0)

        if v == 0:
            new_was_stopped[i] = True

    new_velocities[i] = v

    new_positions = positions[:]
    for i in range(N):
        new_positions[i] = (positions[i] + new_velocities[i]) % L

    pairs = sorted(zip(new_positions, new_velocities,
                       new_was_stopped))
    new_positions = [x for x, v, s in pairs]
    new_velocities = [v for x, v, s in pairs]
    new_was_stopped = [s for x, v, s in pairs]
```

```
return new_positions, new_velocities, new_was_stopped
```

A.13 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. inizializzazione:
 - `initialize_traffic`
 - `initialize_traffic_random`
3. dinamica:
 - `compute_gaps`
 - `nasch_step`
 - `simulate_nasch`
4. analisi:
 - `order_parameter`
 - `velocity_fluctuations`
 - `estimate_wave_speed`
5. diagramma fondamentale:
 - `compute_fundamental_diagram`
 - `asep_flow`
6. visualizzazione:
 - `plot_spacetime_image`
 - `plot_fundamental_diagram`
 - `plot_phase_diagram`
7. varianti:
 - `nasch_slow_to_start_step`
8. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    L = 200
    v_max = 5
    p = 0.3

    # regime fluido
    res_low = simulate_nasch(L=L, N=20, v_max=v_max, p=p,
                             T=300, T_burn=50)
    plot_spacetime_image(res_low["history_positions"], L,
                         title="Regime fluido (rho = 0.1)")

    # regime congestionato
    res_high = simulate_nasch(L=L, N=100, v_max=v_max, p=p,
                              T=300, T_burn=50)
    plot_spacetime_image(res_high["history_positions"], L,
                         title="Regime congestionato (rho = 0.5)")
```

```

# diagramma fondamentale
rho_vals, flows, _ = compute_fundamental_diagram(
    L=L, v_max=v_max, p=p, T=2000, T_burn=500
)
plot_fundamental_diagram(rho_vals, flows,
    label=f"p={p}",
    title="Diagramma fondamentale")

# verifica ASEP
plot_asep_comparison(p=0.3, L=200)

```

A.14 Perché questa appendice è utile

Questa appendice ha due funzioni didattiche principali.

Primo, il confronto tra il caso $v_{\max} = 1$ analitico e la simulazione mostra immediatamente se il codice è corretto: non serve fidarsi della simulazione a occhio, si ha una formula di riferimento.

Secondo, la separazione tra `nasch_step` (la fisica) e `simulate_nasch` (la raccolta dei dati) rende molto semplice modificare il modello per studiare varianti: basta sostituire `nasch_step` con una versione diversa, e tutto il resto rimane uguale.

A.15 Conclusione dell'appendice

La struttura proposta è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il punto metodologico importante è che un automa cellulare stocastico, per quanto semplice, produce una fenomenologia molto ricca: transizioni di fase, onde propaganti, dipendenza non lineare dai parametri. Tutta questa ricchezza emerge da quattro regole che si scrivono in poche righe di codice.